

greyimage

February 17, 2026

0.1 Ejercicio 0

Ejecuta el código de la siguiente celda. Asegurate que la carpeta “images” incluye las imágenes proporcionadas. Examina los resultados que se generaron y analiza brevemente el código proporcionado.

```
[1]: # Código para convertir a gris las imagenes alojadas en la carpeta "images"

import matplotlib
import matplotlib.image as pltim
import matplotlib.pyplot as plt
import numpy as np
import time
import os

folder = "images" # Asegurar que el link anterior se llamada images

def list_jpg_files(folder_path):
    jpg_files = []
    for filename in os.listdir(folder_path):
        if filename.lower().endswith(".jpg"):
            jpg_files.append(filename)
    return jpg_files

def rgb2gray(image):

    imageHeight = len(image)
    imageWidth = len(image[0])
    print(imageHeight, imageWidth)
    greyImage = np.empty((imageHeight, imageWidth), dtype=np.uint8)

    for j in range(imageWidth):
        for i in range(imageHeight):
            greyImage[i][j] = np.int8(image[i][j][0]*0.2989 + image[i][j][1]*0.
↪5870 + image[i][j][2] * 0.1140)
    return greyImage

def showImage(image):
```

```

plt.imshow(image, cmap = matplotlib.cm.Greys_r)
plt.show()

tiempos_ejecución_ejercicio0 = []

# Obtener la lista de imagenes de la carpeta "folder"
jpg_files_list = list_jpg_files(folder)

# Recorrer la lista para convertir a gris
for image_file in jpg_files_list:

    image_file = folder+"/"+image_file
    image = pltim.imread(image_file)
    print(f"Processing image {image_file}")

    start_time = time.time()

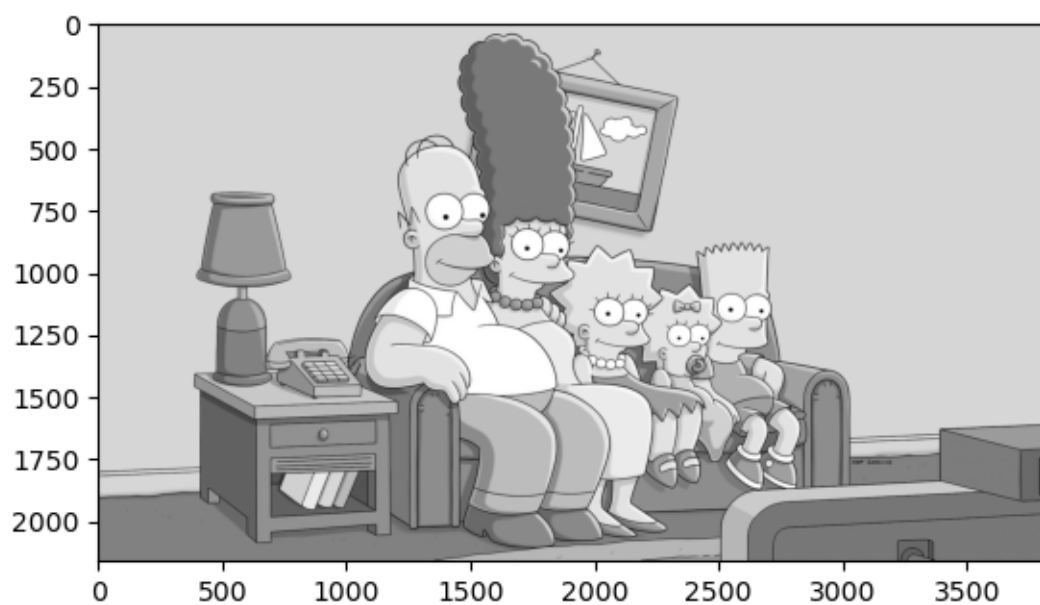
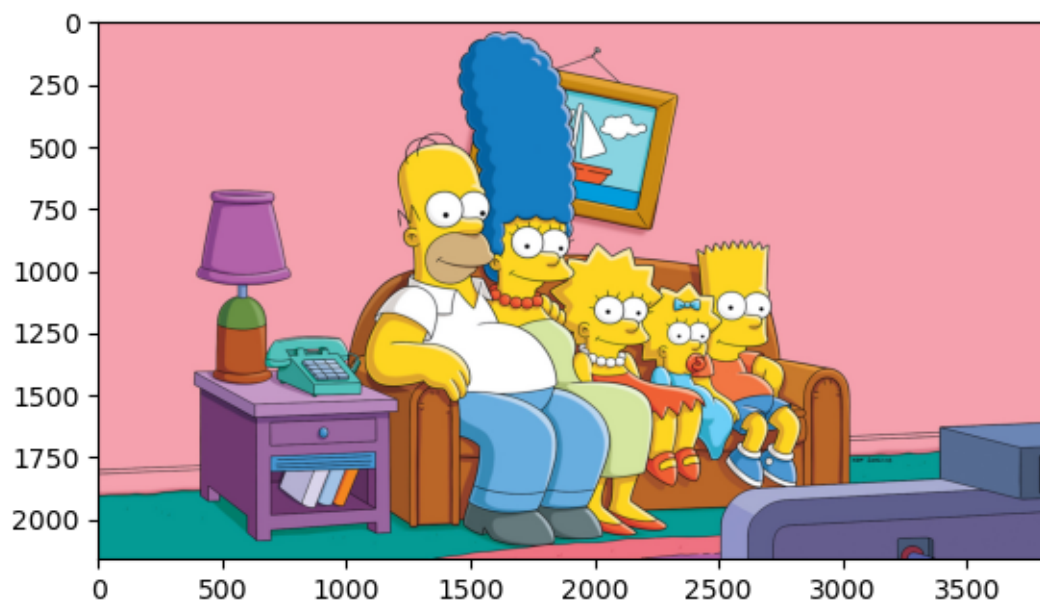
    greyimage = rgb2gray(image)

    end_time = time.time()
    execution_time = end_time - start_time
    tiempos_ejecución_ejercicio0.append(execution_time)
    print(f"Execution time: {execution_time:.6f} seconds")

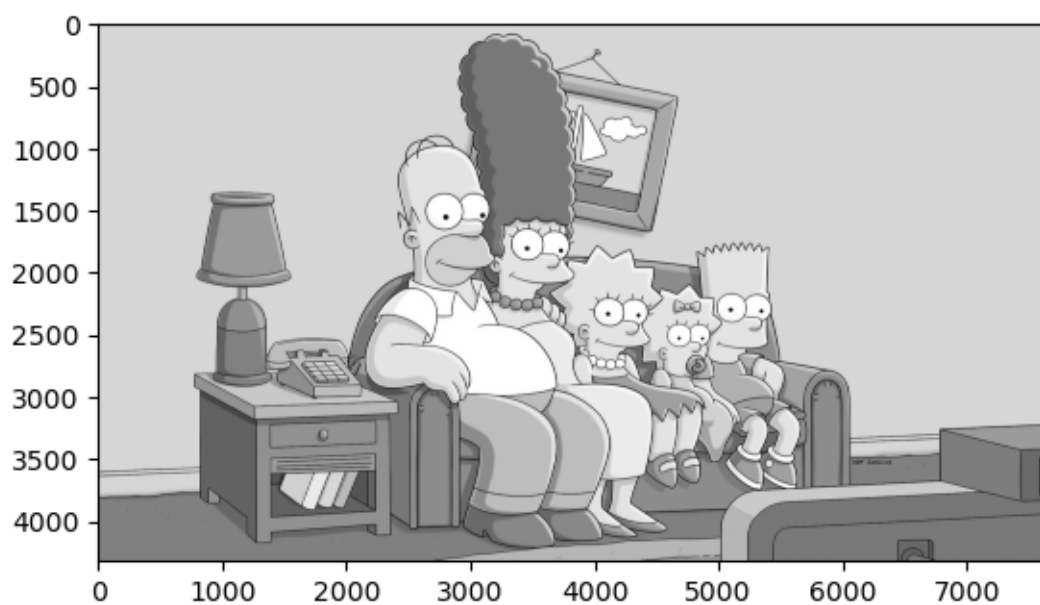
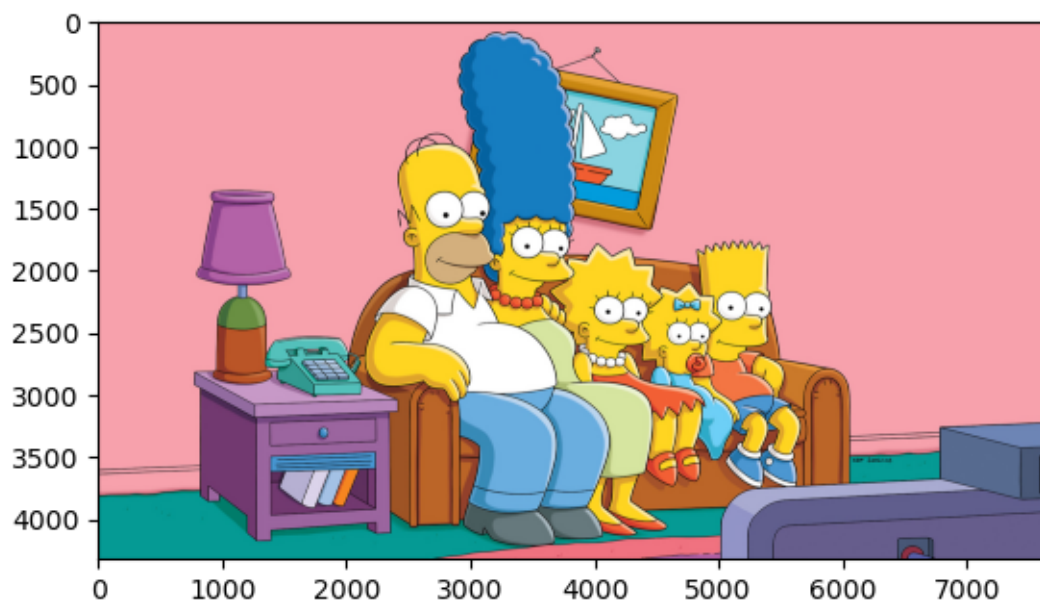
    showImage(image)
    showImage(greyimage)

```

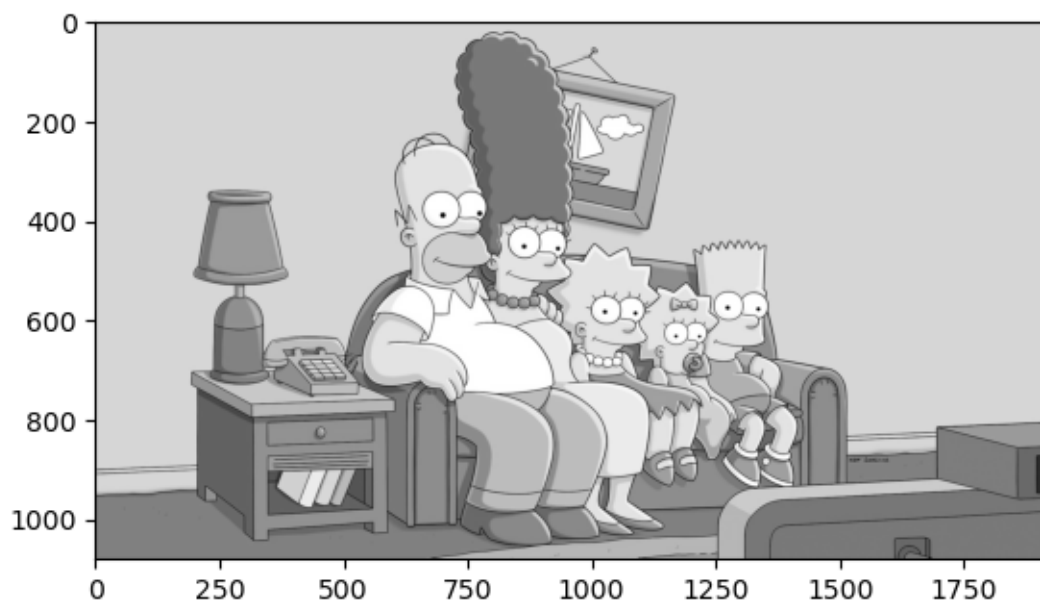
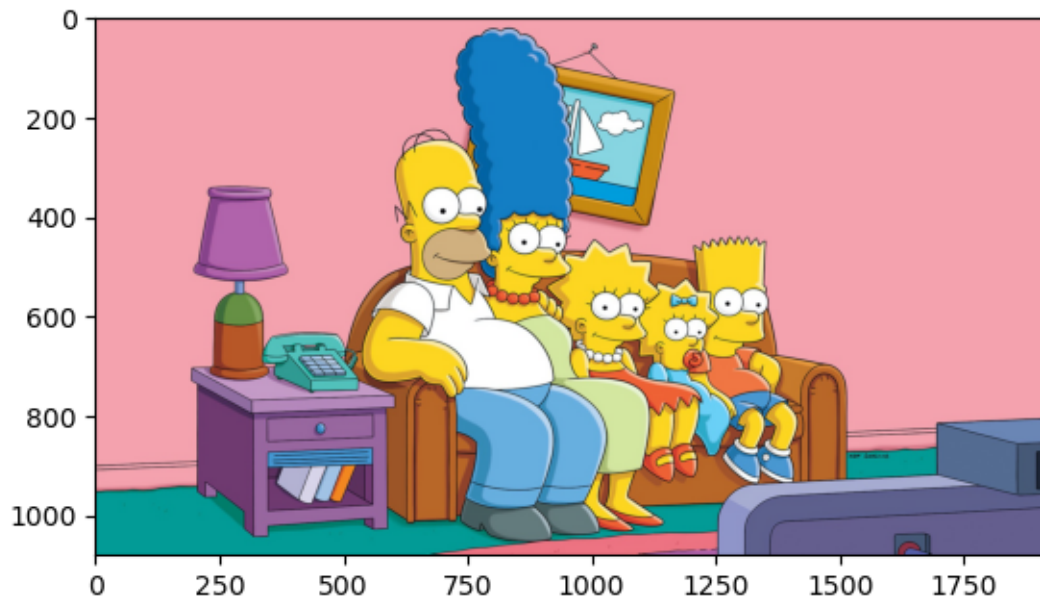
Processing image images/4k.jpg
2160 3840
Execution time: 18.257398 seconds



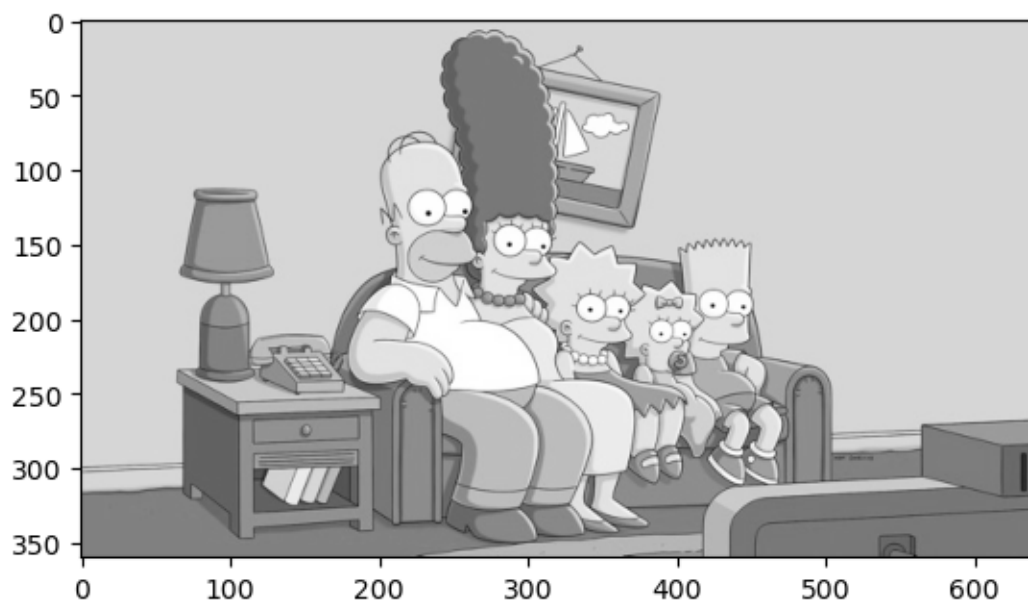
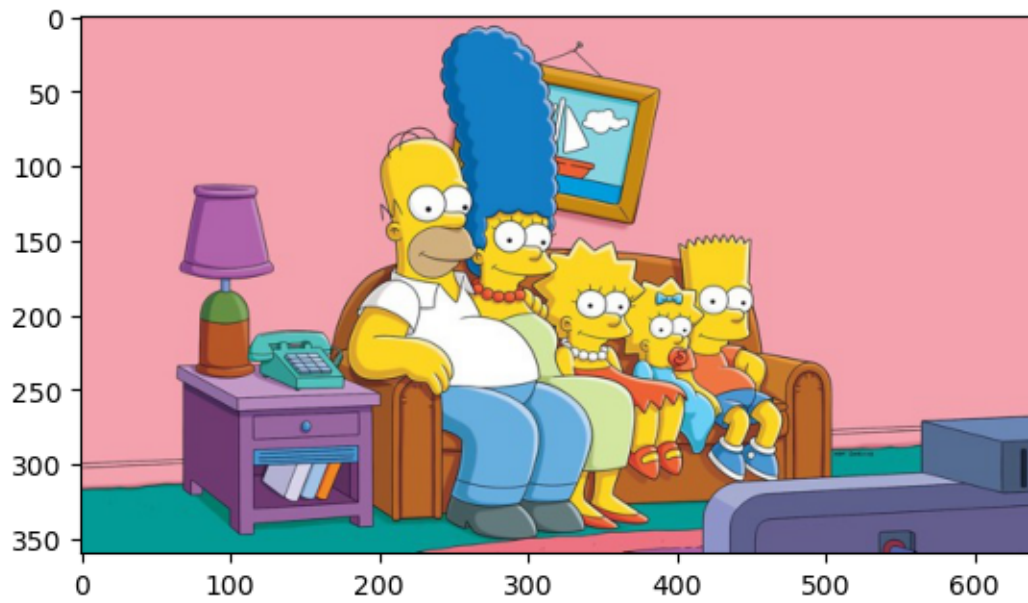
Processing image images/8k.jpg
4320 7680
Execution time: 71.217051 seconds



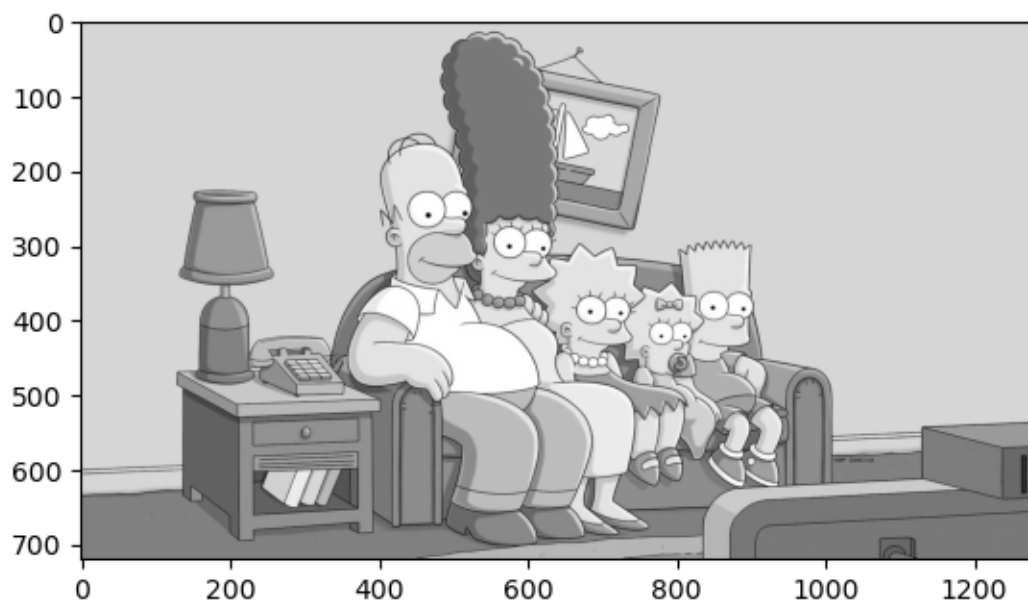
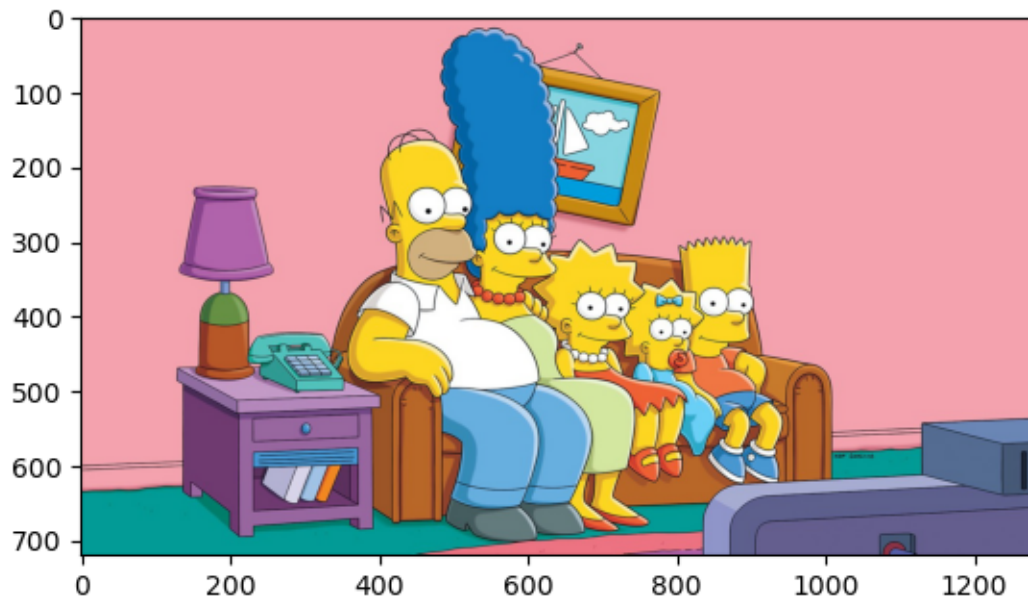
Processing image images/FHD.jpg
 1080 1920
 Execution time: 4.467483 seconds



Processing image images/SD.jpg
360 640
Execution time: 0.527805 seconds



Processing image images/HD.jpg
720 1280
Execution time: 1.999584 seconds



En este caso se generan 5 pares de imágenes true-color - true-bw, que al ser leídas pixel por pixel, leyendo primero columnas y después filas, tarda mucho tiempo en visualizarlas. En concreto:

- Primera imagen, 4k: 18.34 segundos (0.054 fps).
- Segunda imagen, 8k: 78.89 segundos (0.012 fps).
- Tercera imagen FHD: 5.26 segundos (0.190 fps).
- Cuarta imagen, SD: 0.52 segundos (1.93 fps).

- Quinta imagen, HD: 2.09 segundos (0.48 fps).

Es evidente que este método no es asumible para leer imágenes. En el mejor de los casos, conseguimos 1.93 fps (con la imagen de peor calidad, obviamente). En cuanto busquemos algo de calidad en la imagen, esto sería completamente imposible.

0.2 Ejercicio 1

Sabemos que el orden de acceso a los datos es importante. Corrija el código si es necesario para mejorar el rendimiento. Explique sus cambios.

```
[2]: def rgb2gray_ejercicio1(image):

    imageHeight = len(image)
    imageWidth = len(image[0])
    print(imageHeight, imageWidth)
    greyImage = np.empty((imageHeight, imageWidth), dtype=np.uint8)
    # Cambiamos el orden: primero filas y luego columnas.
    for i in range(imageHeight):
        for j in range(imageWidth):
            greyImage[i][j] = np.int8(image[i][j][0]*0.2989 + image[i][j][1]*0.
↪5870 + image[i][j][2] * 0.1140)
    return greyImage

tiempos_ejecución_ejercicio1 = []

# Recorrer la lista para convertir a gris
for image_file in jpg_files_list:

    image_file = folder+"/"+image_file
    image = pltim.imread(image_file)
    print(f"Processing image {image_file}")

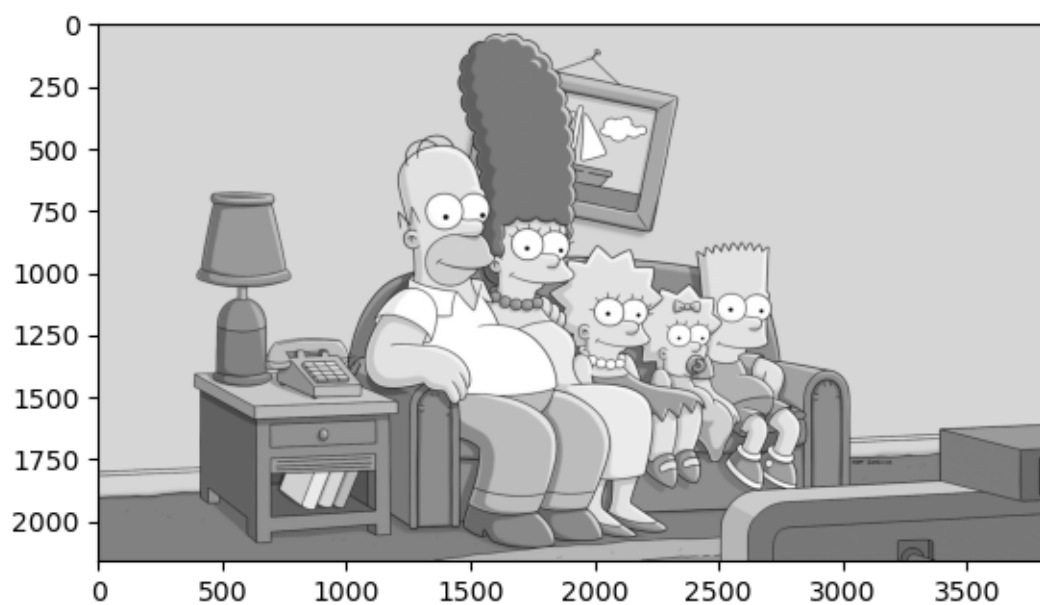
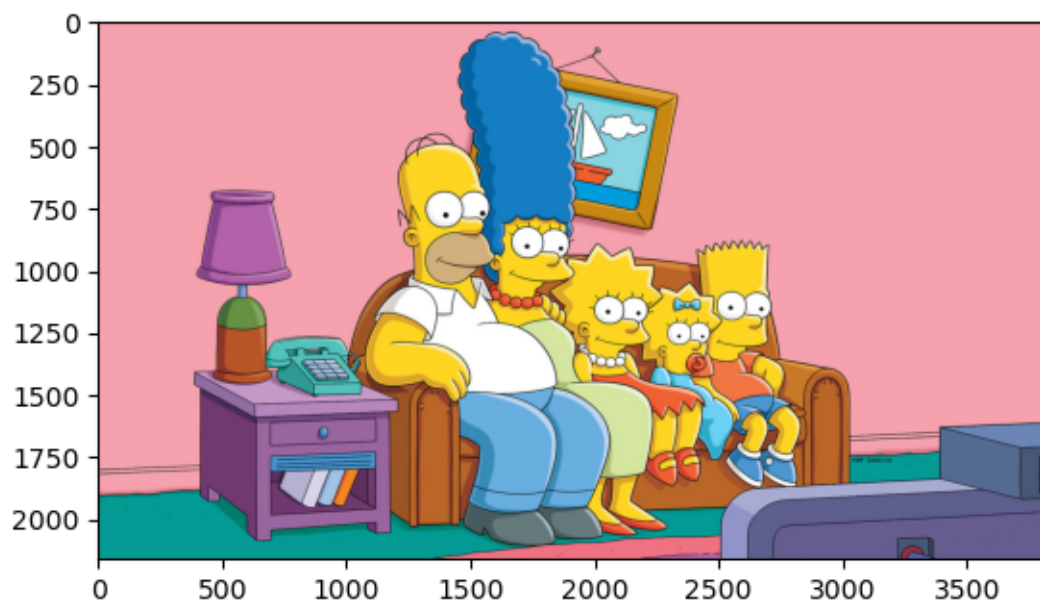
    start_time = time.time()

    greyimage = rgb2gray_ejercicio1(image)

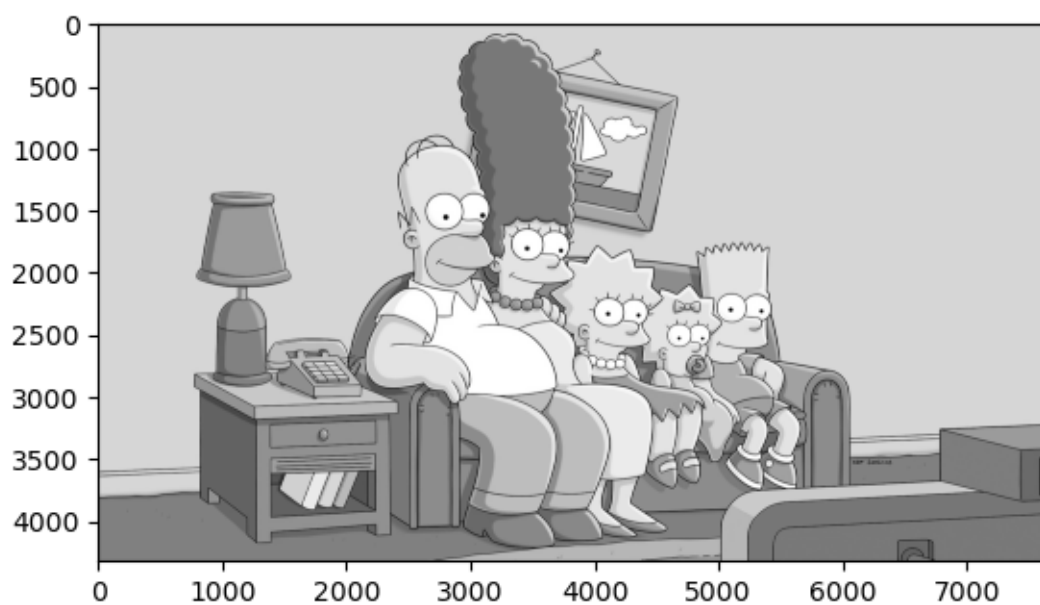
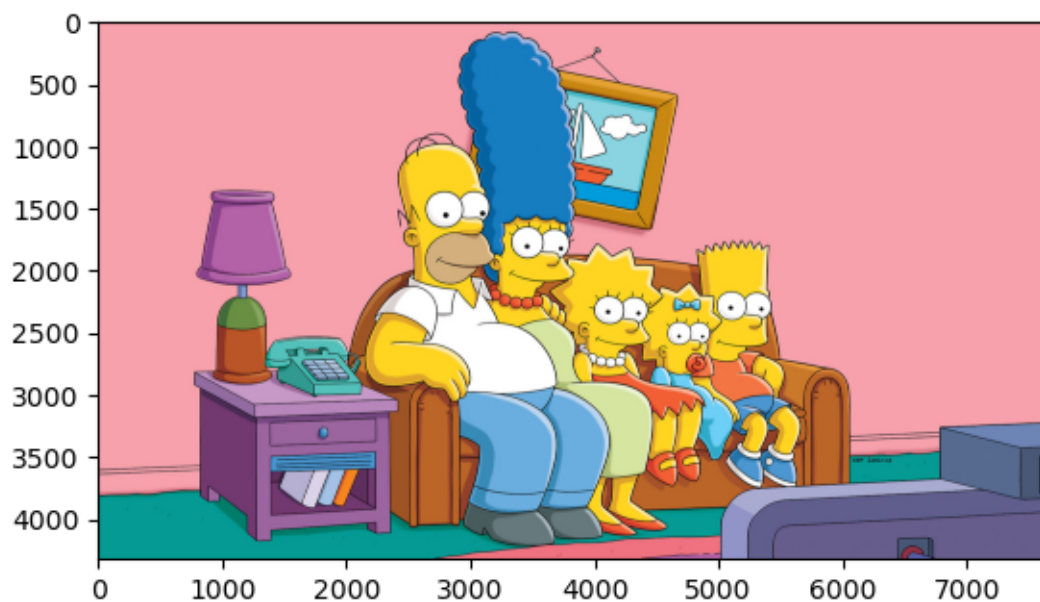
    end_time = time.time()
    execution_time = end_time - start_time
    tiempos_ejecución_ejercicio1.append(execution_time)
    print(f"Execution time: {execution_time:.6f} seconds")

    showImage(image)
    showImage(greyimage)
```

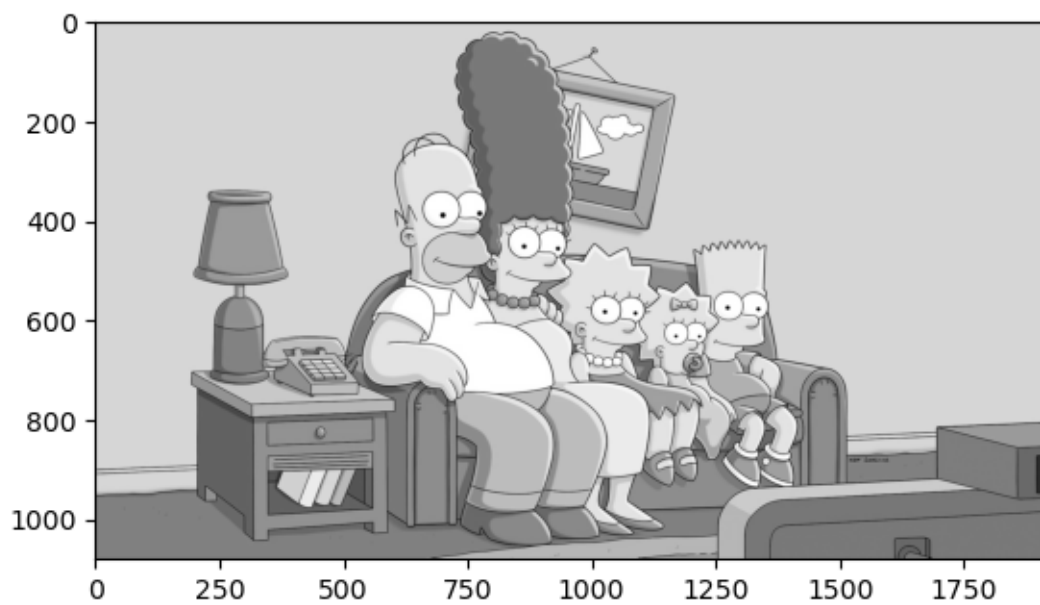
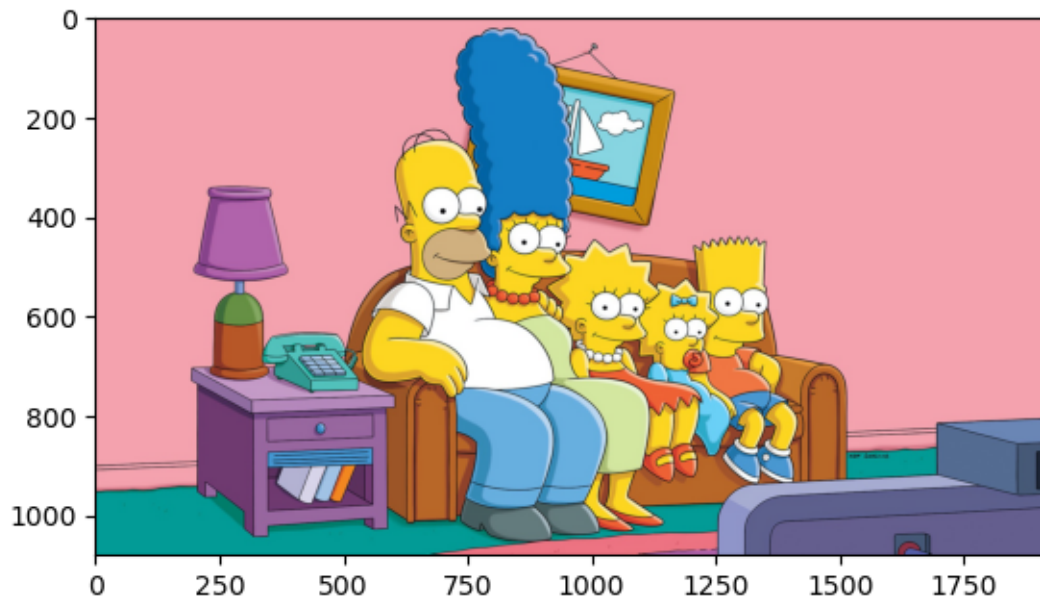
```
Processing image images/4k.jpg
2160 3840
Execution time: 17.677550 seconds
```

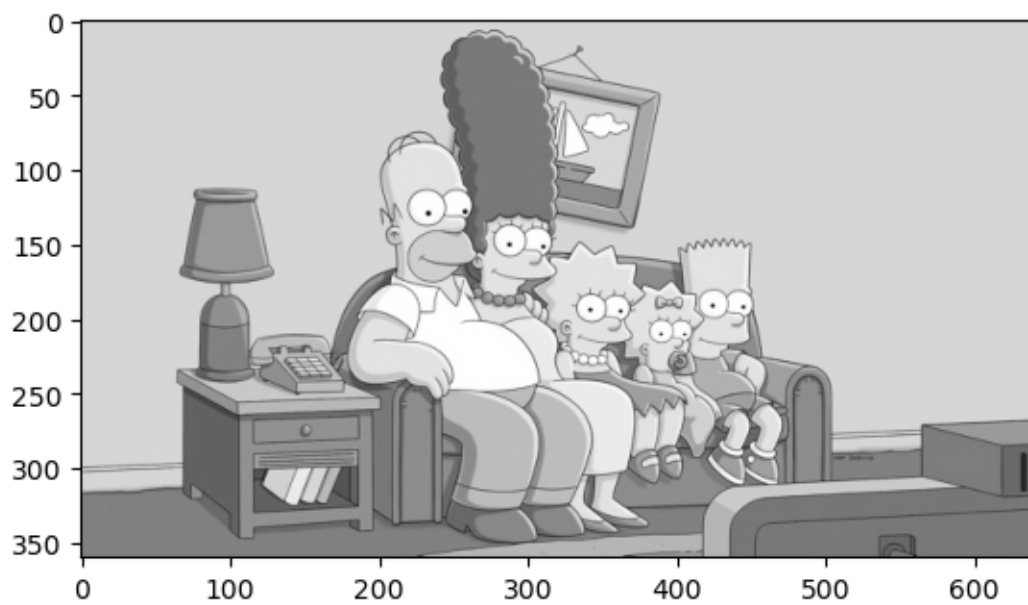
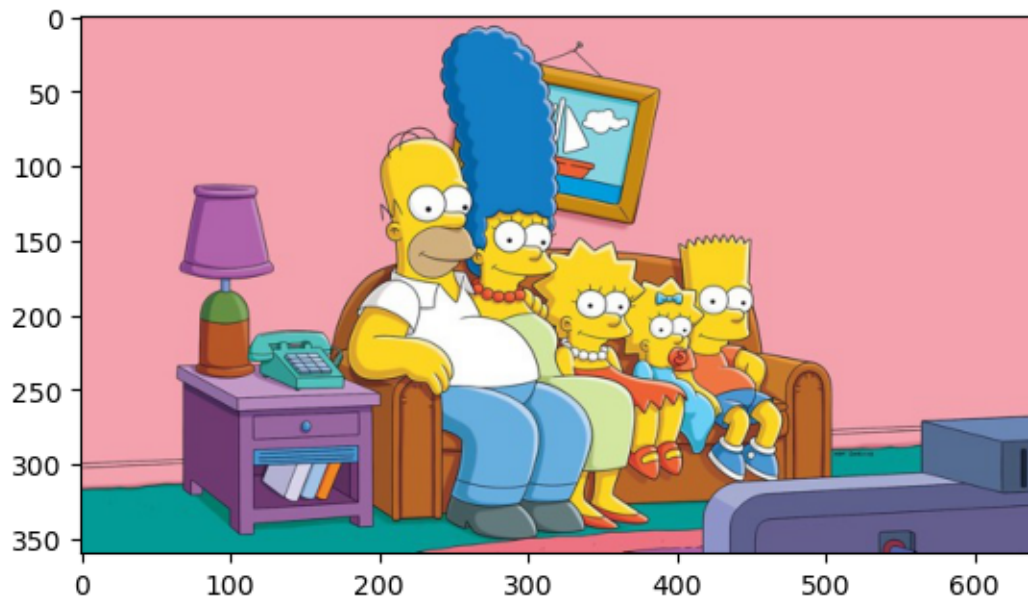
Processing image images/8k.jpg
 4320 7680
 Execution time: 70.621061 seconds



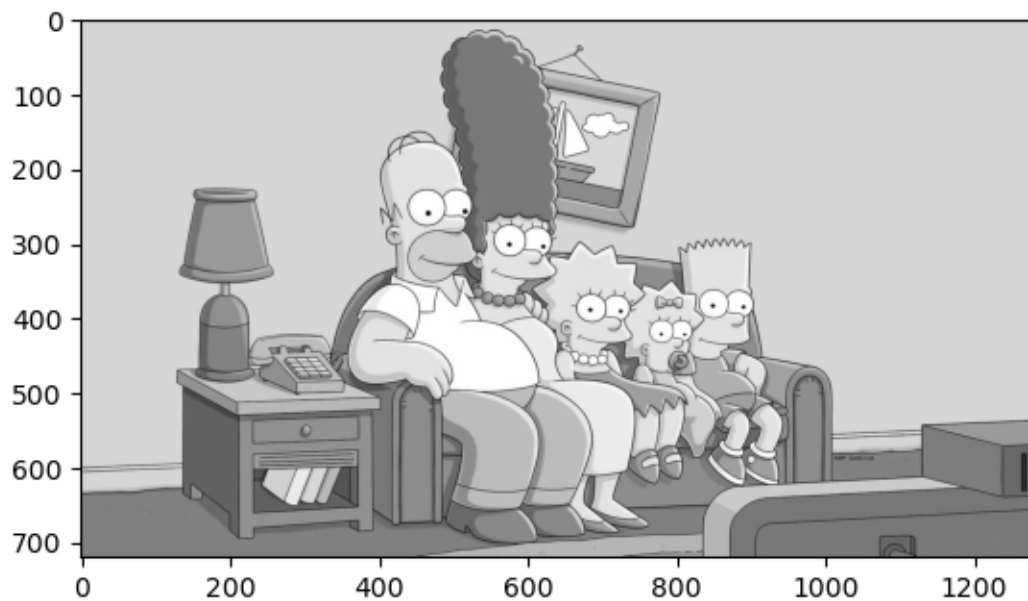
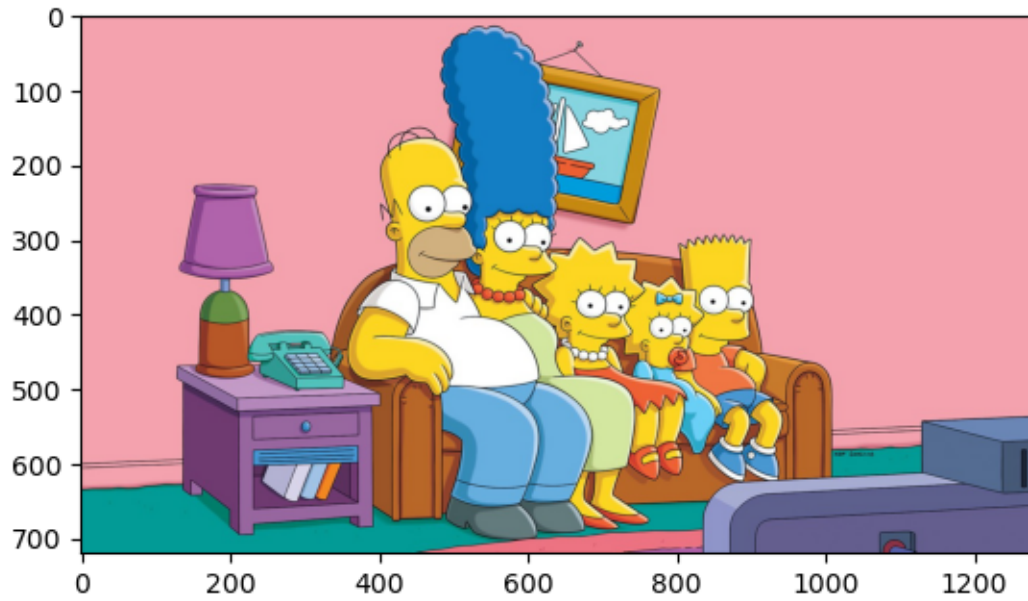
Processing image images/FHD.jpg
1080 1920
Execution time: 4.600020 seconds



Processing image images/SD.jpg
360 640
Execution time: 0.552804 seconds



Processing image images/HD.jpg
720 1280
Execution time: 2.046690 seconds



En este caso se leen filas y después columnas, pero aún así tarda mucho tiempo en visualizarlas. Esto es debido al rendimiento de Python, probablemente culpable de más del 95% del tiempo de ejecución. En concreto:

- Primera imagen, 4k: 18.71 segundos (0.054 fps $\rightarrow$ 0.053 fps) $\rightarrow$ el rendimiento empeora, aunque es algo que podemos esperar al hacer mediciones experimentales.

- Segunda imagen, 8k: 76.57 segundos (0.012 fps -> 0.013 fps) -> cierta mejoría, apenas notable o despreciable.
- Tercera imagen FHD: 4.85 segundos (0.190 fps -> 0.205 fps) -> cierta mejoría, apenas notable.
- Cuarta imagen, SD: 0.51 segundos (1.93 fps -> 1.93 fps) -> cierta mejoría, apenas notable.
- Quinta imagen, HD: 2.25 segundos (0.48 fps -> 0.44 fps) -> el rendimiento empeora, aunque es algo que podemos esperar al hacer mediciones experimentales.

Este método sigue sin ser asumible para leer imágenes. En el mejor de los casos, conseguimos 1.43 fps (con la imagen de peor calidad, obviamente).

0.3 Ejercicio 2

Aplique Numba para la generación de código SIMD. Verifique que se han generado instrucciones SIMD. Complete una tabla con los resultados de tiempo y aceleración para comparar la versión original (con la corrección del ejercicio 1) y la versión SIMD para las imágenes de diferentes resoluciones (SD, HD, FHD, 4k, 8k). Debe incluir una columna con los fps a los que procesaría el programa. Discuta los resultados.

```
[3]: from numba import jit

# Añadimos el decorador de numba para compilar la función.
@jit(nopython=True)
def rgb2gray_ejercicio2(image):
    imageHeight = len(image)
    imageWidth = len(image[0])
    print(imageHeight, imageWidth)
    greyImage = np.empty((imageHeight, imageWidth), dtype=np.uint8)
    # Y mantenemos el orden de ejecución:
    for i in range(imageHeight):
        for j in range(imageWidth):
            greyImage[i][j] = np.int8(image[i][j][0]*0.2989 + image[i][j][1]*0.
↪5870 + image[i][j][2] * 0.1140)
    return greyImage

tiempos_ejecución_ejercicio2 = []

# Primero una llamada en frío.
image = plt.imread('images/SD.jpg')
rgb2gray_ejercicio2(image)

# Recorrer la lista para convertir a gris
for image_file in jpg_files_list:

    image_file = folder+"/"+image_file
    image = plt.imread(image_file)
    print(f"Processing image {image_file}")
```

```

start_time = time.time()

greyimage = rgb2gray_ejercicio2(image)

end_time = time.time()
execution_time = end_time - start_time
tiempos_ejecución_ejercicio2.append(execution_time)
print(f"Execution time: {execution_time:.6f} seconds")

showImage(image)
showImage(greyimage)

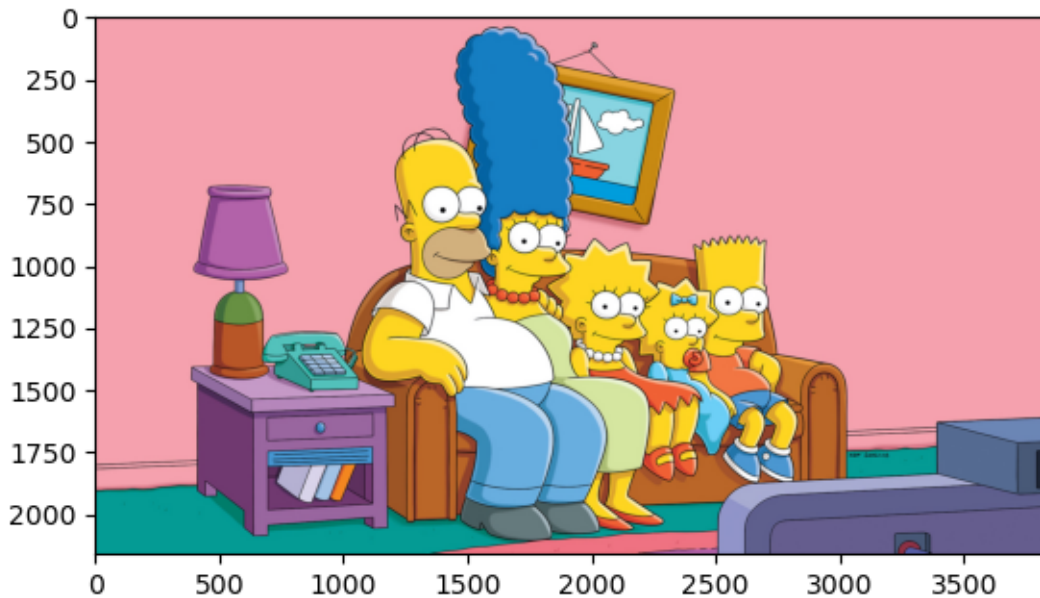
```

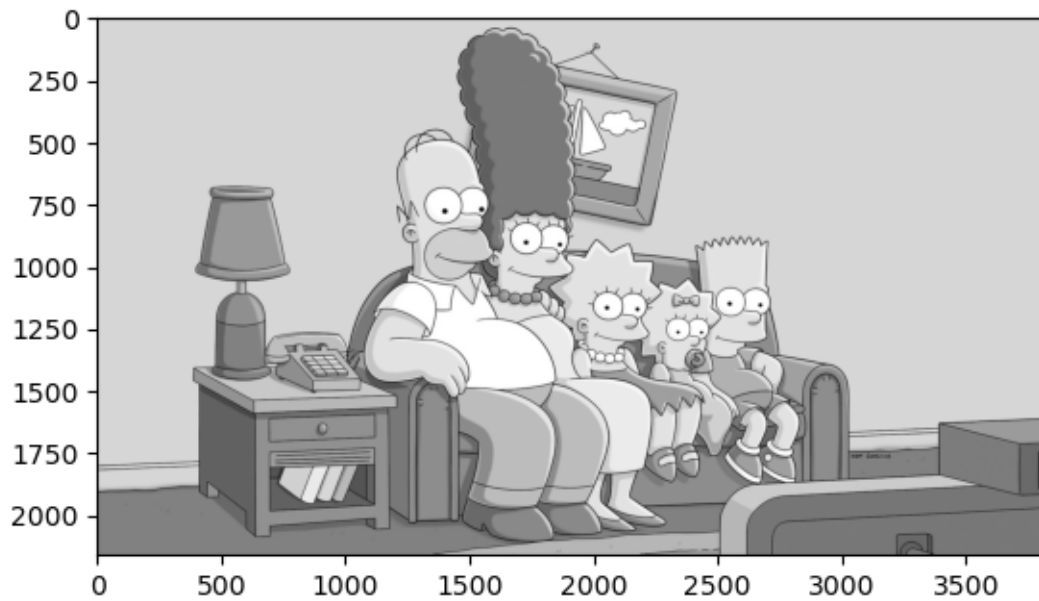
360 640

Processing image images/4k.jpg

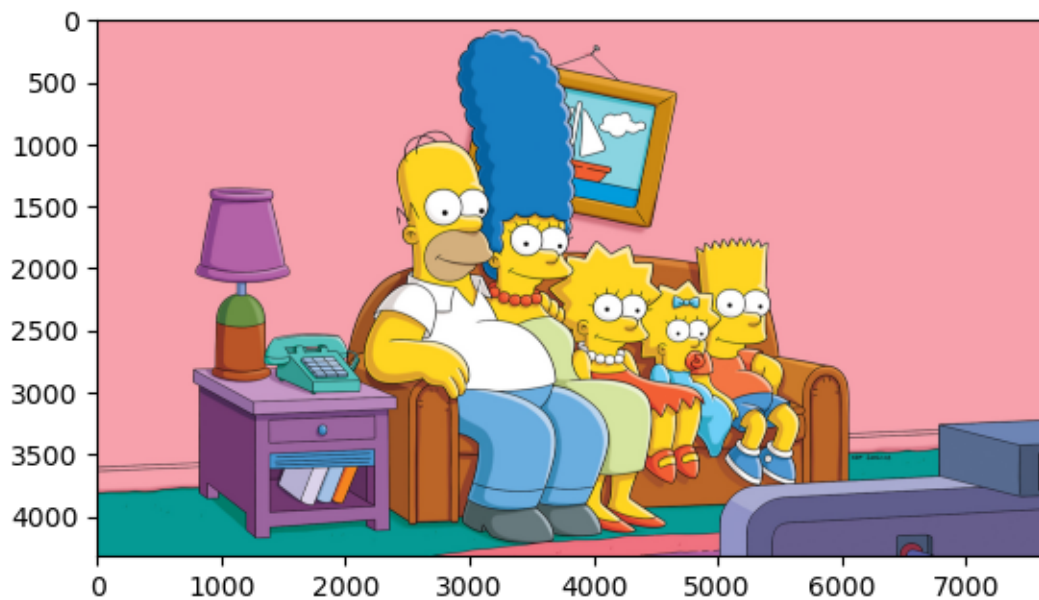
2160 3840

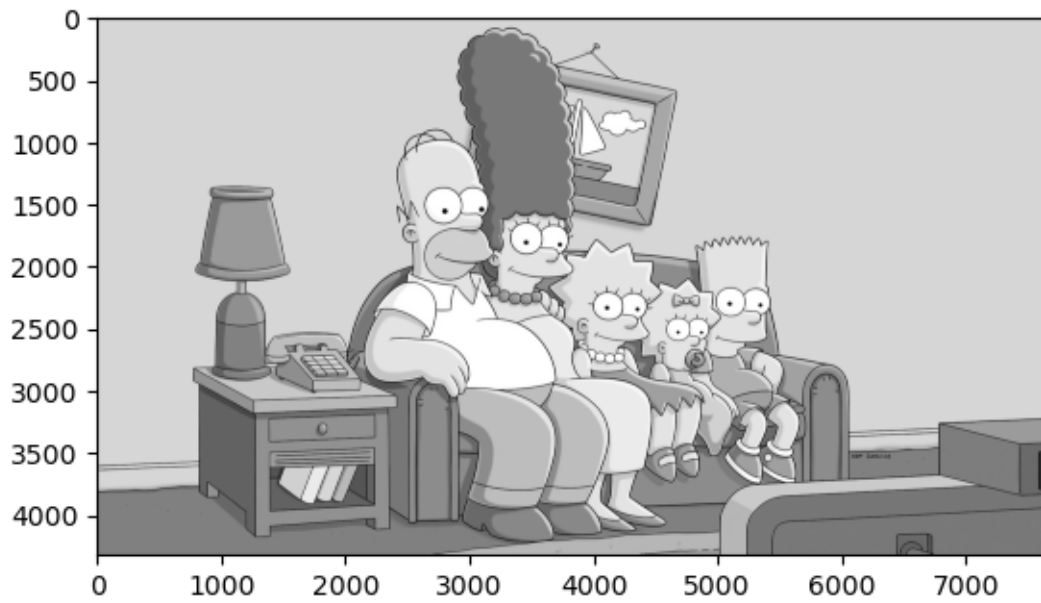
Execution time: 0.005441 seconds



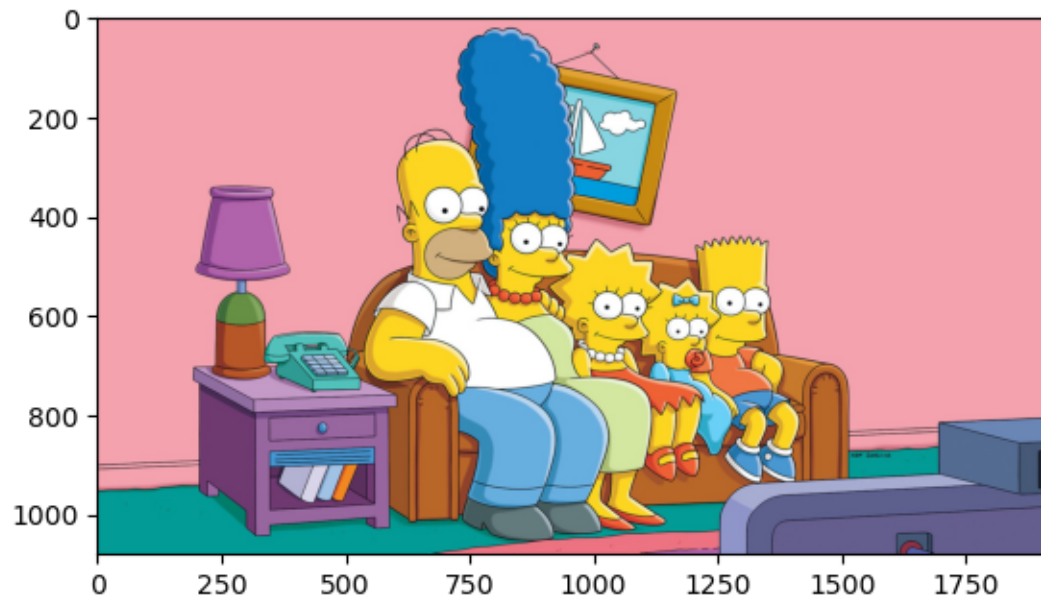


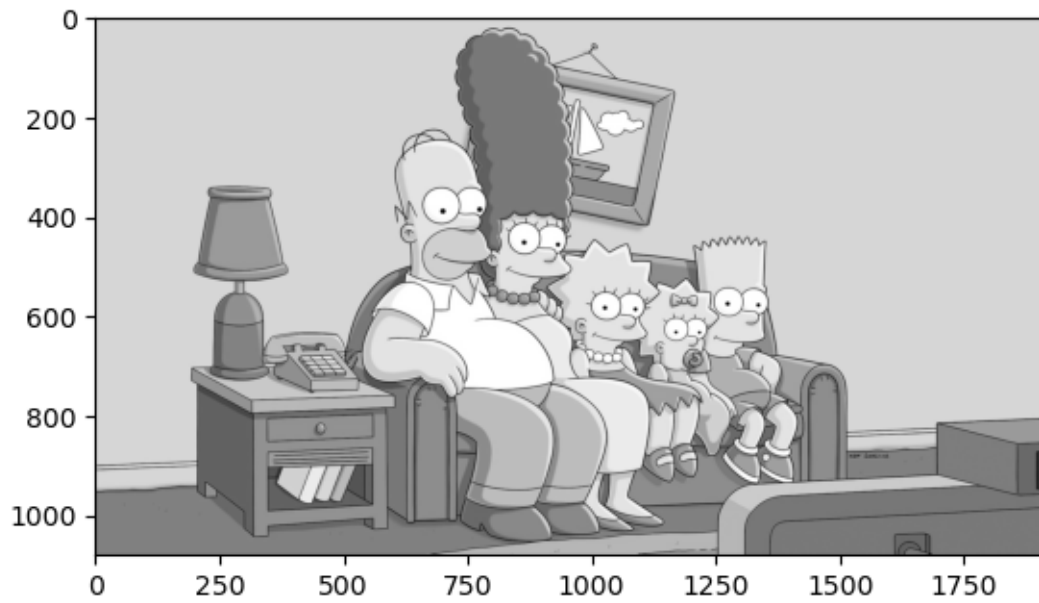
Processing image images/8k.jpg
4320 7680
Execution time: 0.022662 seconds



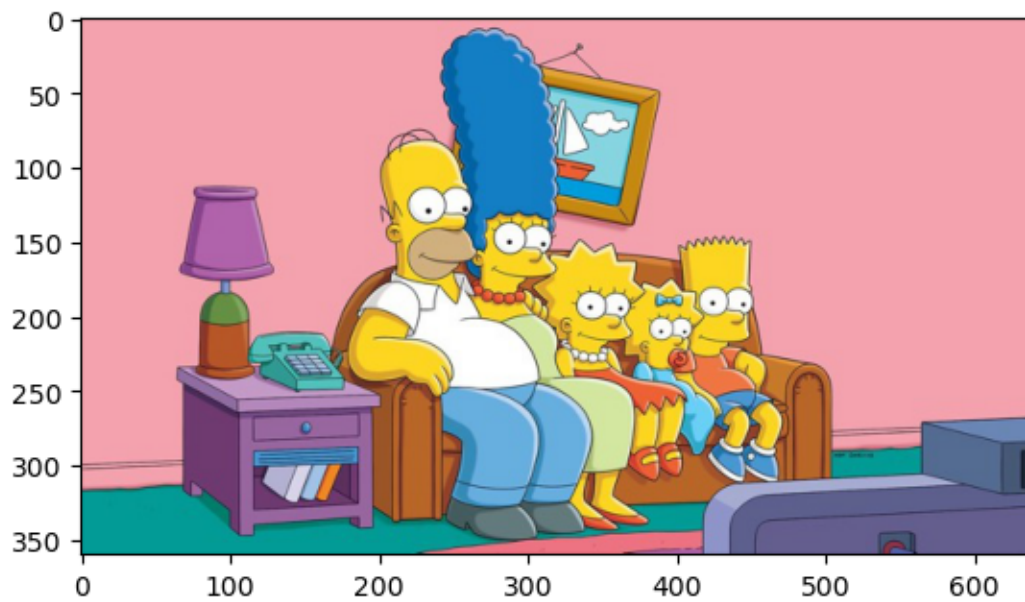


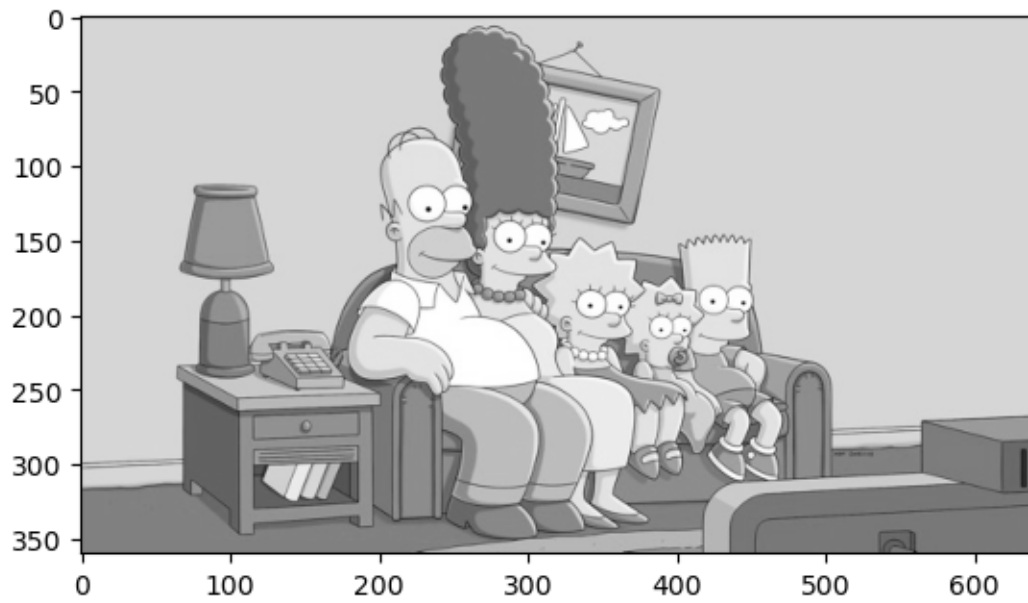
Processing image images/FHD.jpg
1080 1920
Execution time: 0.001626 seconds



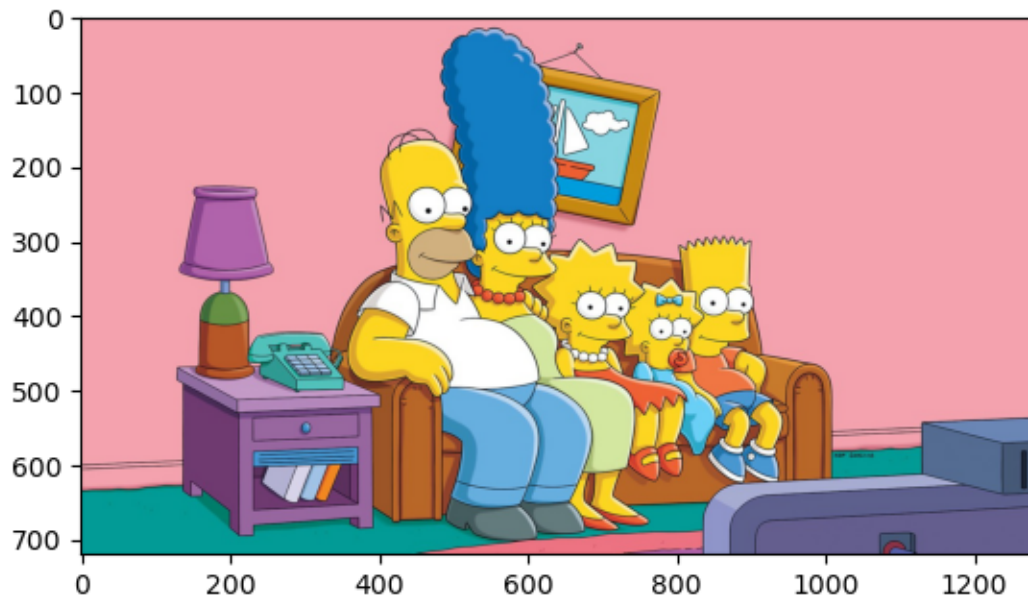


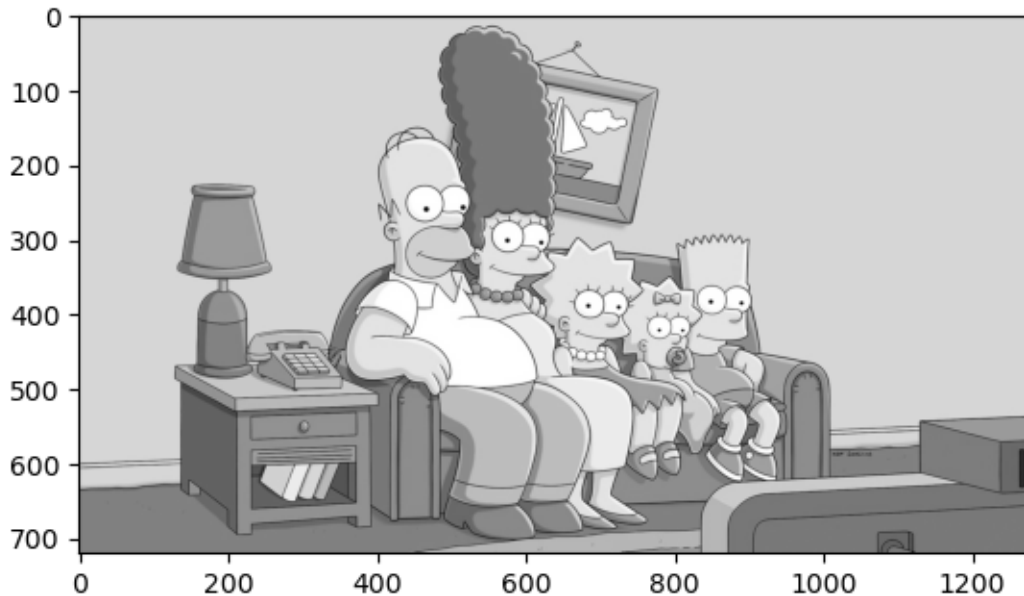
Processing image images/SD.jpg
360 640
Execution time: 0.000940 seconds





Processing image images/HD.jpg
720 1280
Execution time: 0.000851 seconds





Podemos ver también si se han generado instrucciones SIMD, lo que demostraría que efectivamente el código se está compilando. Para ello, podemos copiar la función que veíamos en el notebook *simd.ipynb*.

```
[4]: def find_instr(func, keyword, sig=0, limit=5):
    count = 0
    for l in func.inspect_asm(func.signatures[sig]).split('\n'):
        if keyword in l:
            count += 1
            print(l)
            if count >= limit:
                break
    if count == 0:
        print('No instructions found')
```

Y comprobar si existe algún tipo de instrucción de este tipo, como la de multiplicación, que sabemos que se tendrá que estar por fuerza si se está resolviendo bien el ejercicio.

```
[5]: print('float32:')
find_instr(rgb2gray_ejercicio2, keyword='fmul', sig=0) # la instrucción en mac,
↳ como veíamos en el ejercicio 2, es 'fmul'
```

```
float32:
    fmul.2d v4, v4, v0
    fmul.2d v5, v5, v0
    fmul.2d v6, v6, v0
    fmul.2d v16, v16, v0
    fmul.2d v18, v18, v0
```

Efectivamente, existe esta instrucción, lo que indica que el código se está compilando.

```
[6]: import pandas as pd

# En primer lugar necesitamos añadir cada tipo de imagen como fila.
# Aprovechamos para añadir los tiempos del ejercicio 0.

my_dict = {}

imagenes = ['4k', '8k', 'FHD', 'SD', 'HD']
for a, b in zip(imagenes, tiempos_ejecución_ejercicio0):
    my_dict[a] = b

df = pd.DataFrame.from_dict(my_dict, orient='index', columns=['Tiempo sin_orden'])
df['fps_0'] = 1/df['Tiempo sin orden']

# Segunda columna y sus fps asociados:

df['Tiempo con orden'] = tiempos_ejecución_ejercicio1
df['fps_1'] = 1/df['Tiempo con orden']

df['Aceleración al corregir índices (%)'] = np.round(((df['Tiempo sin orden'] / df['Tiempo con orden']) * 100) - 100, 2)

# Tercera columna y sus fps asociados:

df['Tiempo con compilacion'] = tiempos_ejecución_ejercicio2
df['fps_2'] = 1/df['Tiempo con compilacion']

df['Aceleración al compilar (%)'] = np.round(((df['Tiempo con orden'] / df['Tiempo con compilacion']) * 100) - 100, 2)

df
```

```
[6]:
```

	Tiempo sin orden	fps_0	Tiempo con orden	fps_1 \
4k	18.257398	0.054772	17.677550	0.056569
8k	71.217051	0.014042	70.621061	0.014160
FHD	4.467483	0.223840	4.600020	0.217390
SD	0.527805	1.894640	0.552804	1.808959
HD	1.999584	0.500104	2.046690	0.488594

	Aceleración al corregir índices (%)	Tiempo con compilacion	fps_2 \
4k	3.28	0.005441	183.791420
8k	0.84	0.022662	44.126879
FHD	-2.88	0.001626	615.090776
SD	-4.52	0.000940	1063.734213

HD	-2.30	0.000851	1175.204259
----	-------	----------	-------------

	Aceleración al compilar (%)
4k	324798.20
8k	311528.70
FHD	282842.98
SD	58703.65
HD	240427.88

Es evidente que el rendimiento mejora al compilar. Y ni si quiera hemos ejecutado paralelamente. Pero es curioso ver que con este método podríamos leer imágenes a 1097 frames por segundo a HD, una calidad de imagen más que aceptable y una fluidez imperceptible para el humano. Jugando videojuegos cuesta reconocer la diferencia entre 120 y 240 fps.

0.4 Ejercicio 3

Aplique Numba para generar código paralelo con threads y amplie la tabla de resultados. ¿Cuánto ha mejorado el rendimiento de la aplicación? Discuta los resultados en función del número de cores del entorno de ejecución empleado.

```
[7]: from numba import prange

@jit(nopython=True, parallel=True)
def rgb2gray_ejercicio3(image):
    imageHeight = len(image)
    imageWidth = len(image[0])
    print(imageHeight, imageWidth)
    greyImage = np.empty((imageHeight, imageWidth), dtype=np.uint8)
    # Y mantenemos el orden de ejecución:
    for i in prange(imageHeight):
        for j in range(imageWidth):
            greyImage[i][j] = np.int8(image[i][j][0]*0.2989 + image[i][j][1]*0.
↪5870 + image[i][j][2] * 0.1140)
    return greyImage

tiempos_ejecución_ejercicio3 = []

# Primero una llamada en frío.
image = plt.imread('images/SD.jpg')
rgb2gray_ejercicio3(image)

# Recorrer la lista para convertir a gris
for image_file in jpg_files_list:

    image_file = folder+"/"+image_file
    image = plt.imread(image_file)
    print(f"Processing image {image_file}")
```

```

start_time = time.time()

greyimage = rgb2gray_ejercicio3(image)

end_time = time.time()
execution_time = end_time - start_time
tiempos_ejecución_ejercicio3.append(execution_time)
print(f"Execution time: {execution_time:.6f} seconds")

showImage(image)
showImage(greyimage)

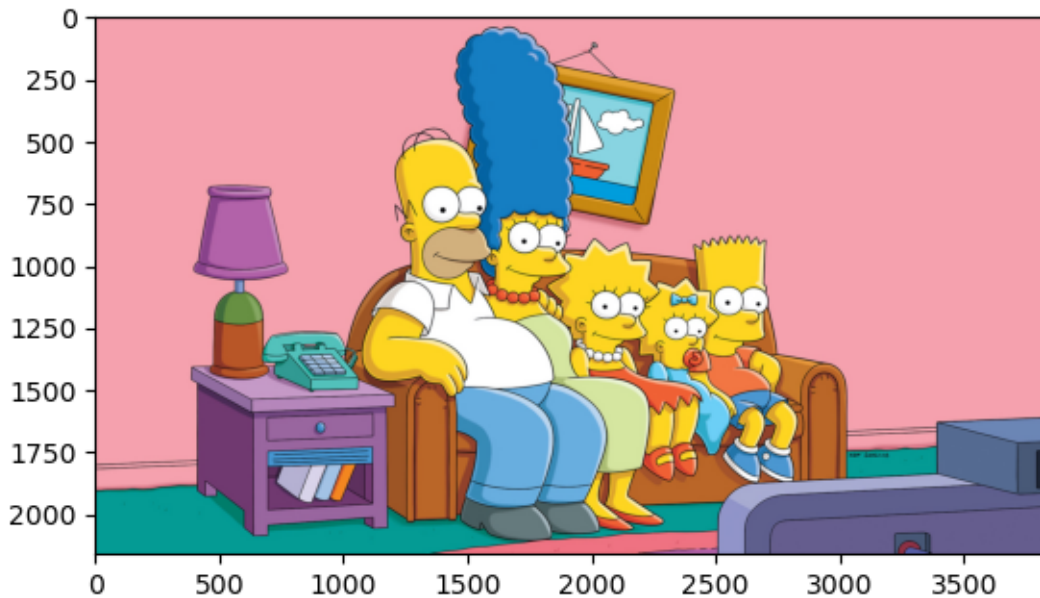
```

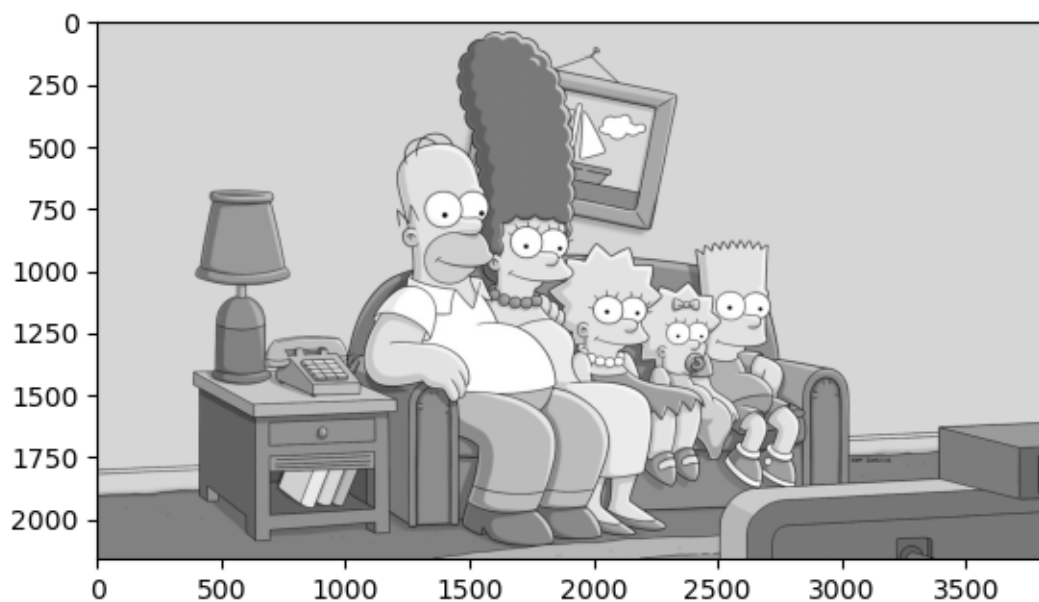
360 640

Processing image images/4k.jpg

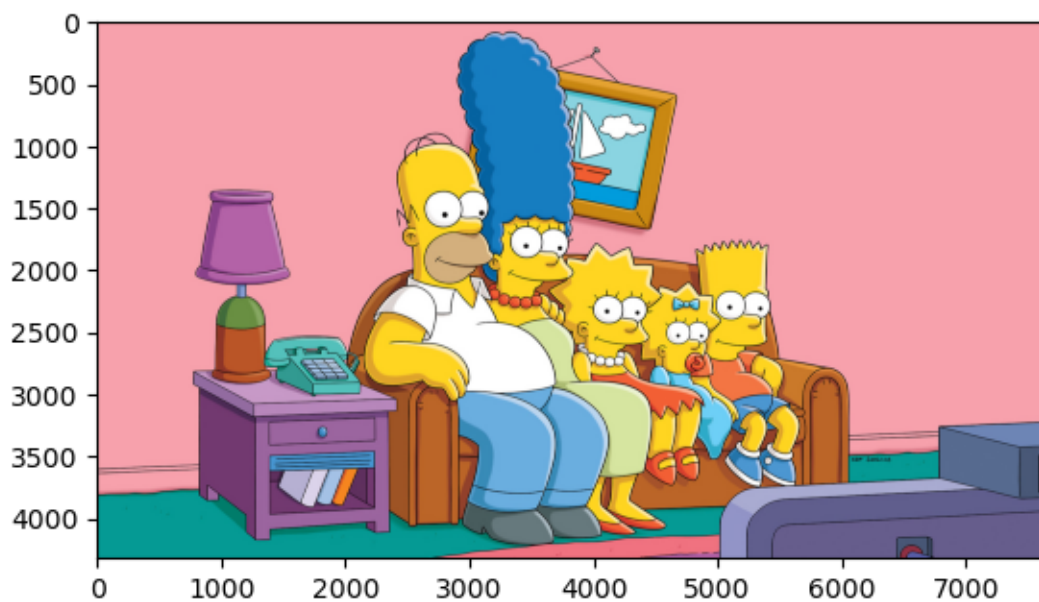
2160 3840

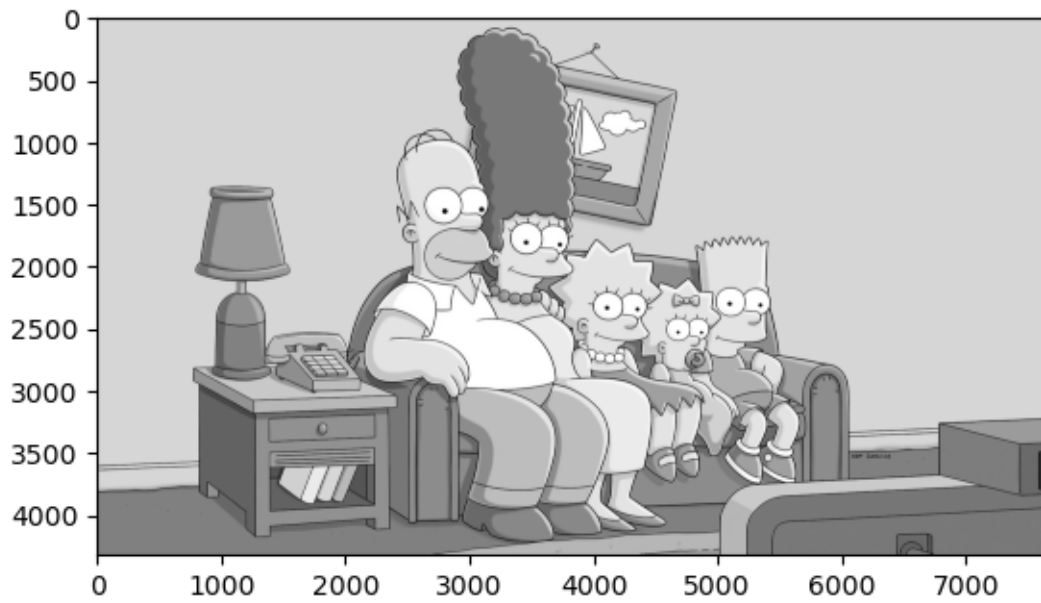
Execution time: 0.001219 seconds



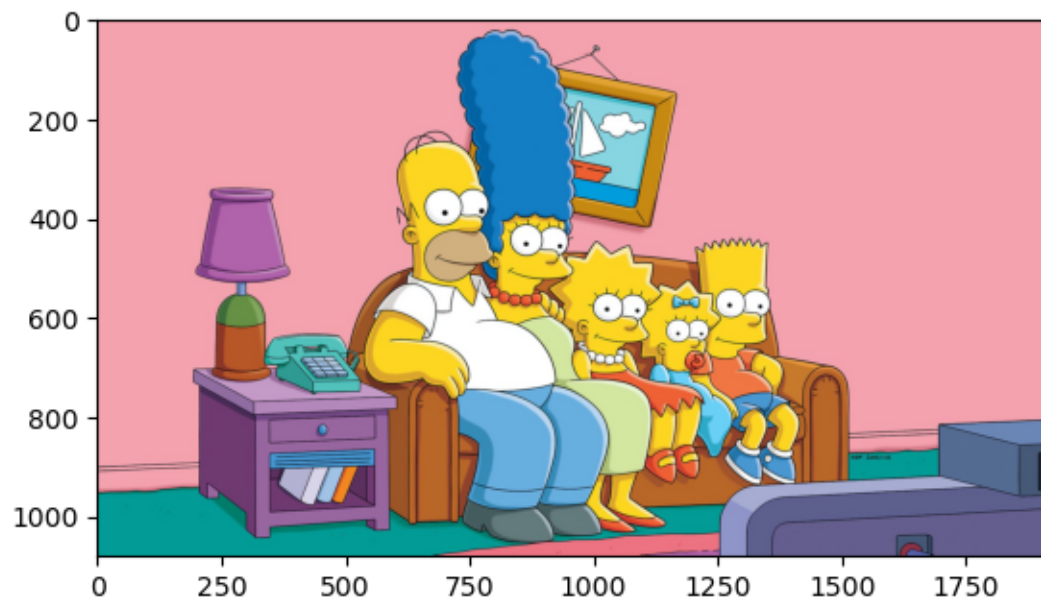


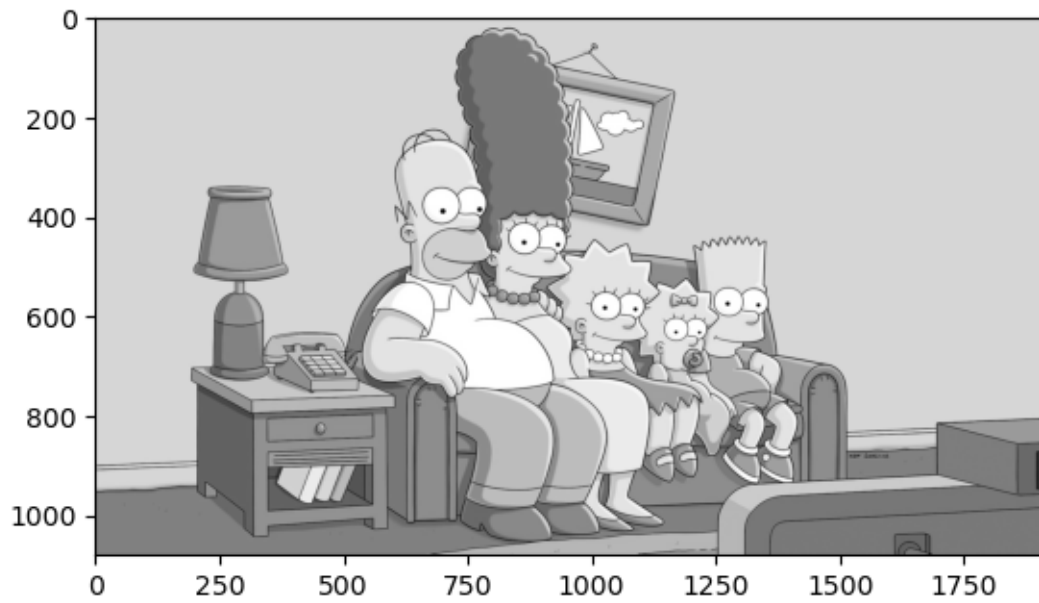
Processing image images/8k.jpg
4320 7680
Execution time: 0.004937 seconds



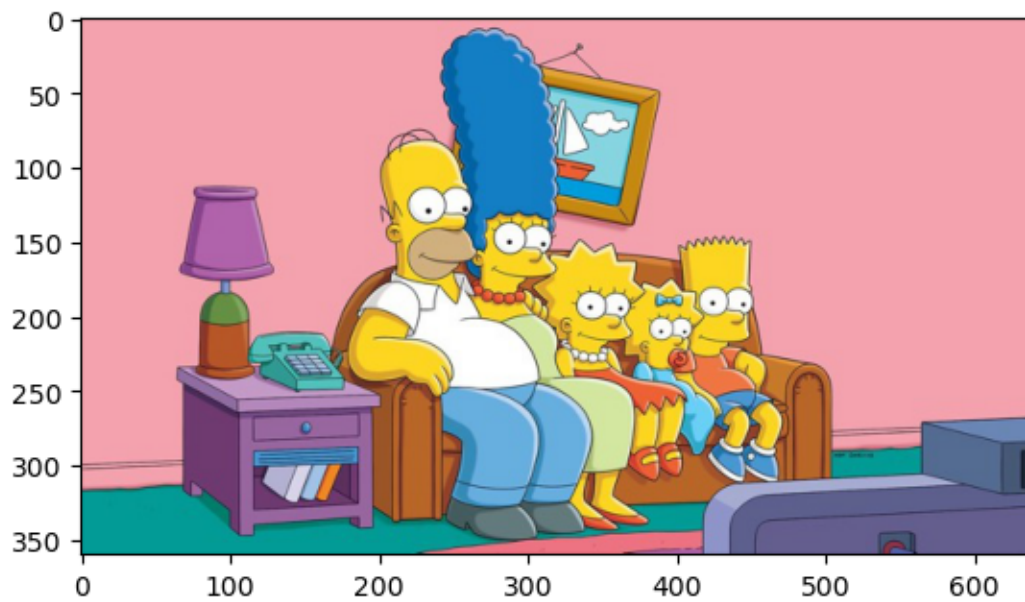


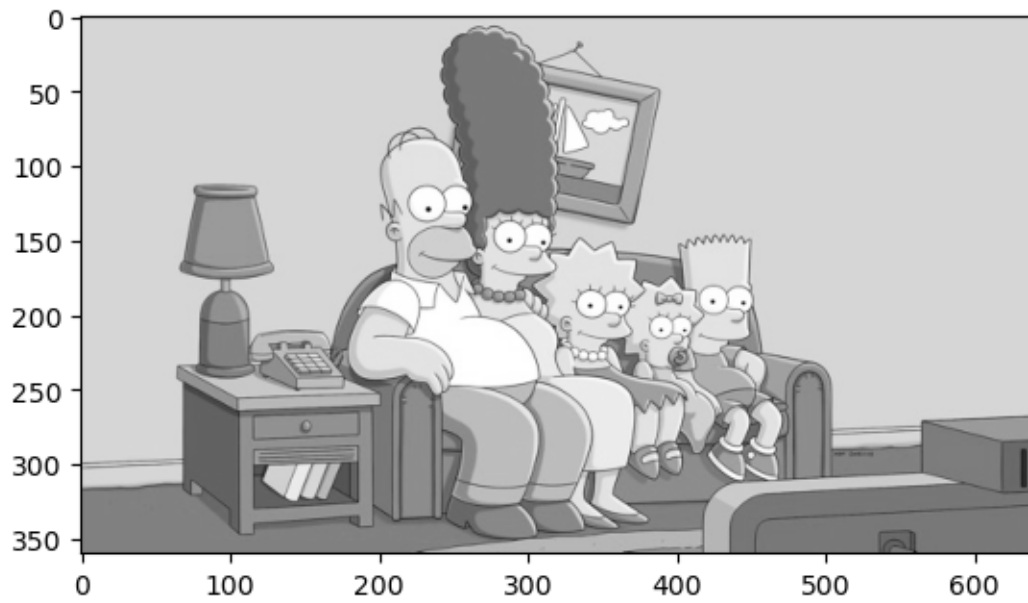
Processing image images/FHD.jpg
1080 1920
Execution time: 0.000632 seconds



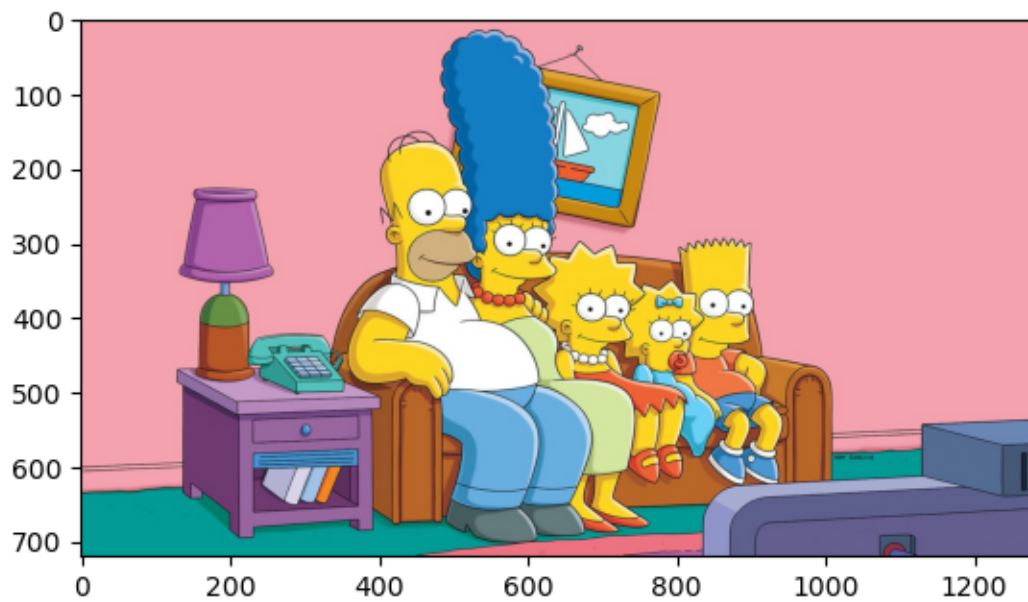


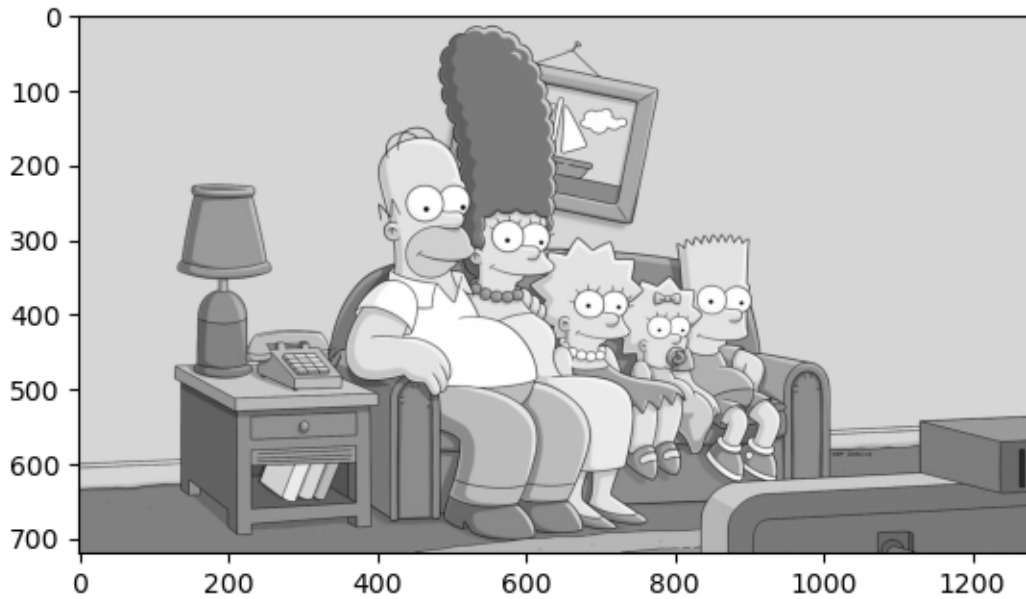
Processing image images/SD.jpg
360 640
Execution time: 0.000872 seconds





Processing image images/HD.jpg
720 1280
Execution time: 0.000602 seconds





```
[8]: df['Tiempo con paralelización'] = tiempos_ejecución_ejercicio3
df['fps_3'] = 1/df['Tiempo con paralelización']

df['Aceleración al paralelizar (%)'] = np.round(((df['Tiempo con compilacion'] /
↪ df['Tiempo con paralelización']) * 100) - 100, 2)

df
```

```
[8]:
```

	Tiempo sin orden	fps_0	Tiempo con orden	fps_1 \
4k	18.257398	0.054772	17.677550	0.056569
8k	71.217051	0.014042	70.621061	0.014160
FHD	4.467483	0.223840	4.600020	0.217390
SD	0.527805	1.894640	0.552804	1.808959
HD	1.999584	0.500104	2.046690	0.488594

	Aceleración al corregir índices (%)	Tiempo con compilacion	fps_2 \
4k	3.28	0.005441	183.791420
8k	0.84	0.022662	44.126879
FHD	-2.88	0.001626	615.090776
SD	-4.52	0.000940	1063.734213
HD	-2.30	0.000851	1175.204259

	Aceleración al compilar (%)	Tiempo con paralelización	fps_3 \
4k	324798.20	0.001219	820.482003
8k	311528.70	0.004937	202.554885
FHD	282842.98	0.000632	1582.159185

SD	58703.65	0.000872	1146.611263
HD	240427.88	0.000602	1661.110495

Aceleración al paralelizar (%)	
4k	346.42
8k	359.03
FHD	157.22
SD	7.79
HD	41.35

Es obvio que, en función de la máquina desde la que se ejecute este código, la tabla final de pandas (la que se muestra justo encima), puede ser muy diferente. Por ello, hemos probado a ejecutar todas las celdas desde 0 desde nuestras dos máquinas. Hemos guardado los resultados aquí:

Resultados en Mac M4 (10 cores):

- La aceleración resultado de paralelizar en este caso oscila entre el 150% y el 400% en el mejor de los casos. Lo lógico sería pensar que el rendimiento al pasar de 1 core a 10 cores es x10, pero eso no ocurre ni en el mejor de los casos. Normalmente esto se debe a que el problema no es absolutamente paralelizable. Se “pierde” el tiempo en otras subtareas, como puede ser la creación de los hilos (a nivel sistema operativo), la lectura de la imagen (siempre tarda un tiempo fijo) o la medición de los tiempos. Todas esas subtareas consumen recursos y solo pueden ser ejecutadas desde un procesador (a priori).

	Tiempo sin orden	fps_0	Tiempo con orden	fps_1	Aceleración al corregir índices (%)	Tiempo con compilación	fps_2	Aceleración al compilar (%)	Tiempo con paralelización	fps_3	Aceleración al paralelizar (%)
4k	17.897878	0.055873	18.126352	0.055168	-1.26	0.005678	176.120260	319141.78	0.001199	834.189340	373.65
8k	70.166415	0.014252	69.767147	0.014333	0.57	0.022083	45.283612	315830.84	0.005223	191.450794	322.78
FHD	4.460609	0.224185	4.413573	0.226574	1.07	0.001620	617.172454	272293.57	0.000657	1521.880987	146.59
SD	0.585140	1.708992	0.479779	2.084294	21.96	0.000165	6052.386724	290280.66	0.000370	2702.515464	-55.35
HD	2.178915	0.458944	1.960939	0.509960	11.12	0.000989	1011.406800	198230.70	0.000227	4405.781513	335.61

Resultados en i7-13620H (16 cores):

- En el caso de intel vemos que la mejora en rendimiento es similar que en Macintosh oscilando entre el 100% y el 570%. También podemos observar que a la hora de usar las versiones mejoradas con SIMD y paralelizada. En las menos costosas, las imágenes SD y HD, el tiempo es tan reducido que el programa registra un tiempo 0, instantaneo, de ahí los “NaN’s” e “inf’s” que vemos en los resultados del intel. No es sorprendente que esto sea así en una máquina de 16 cores, pues las instrucciones SIMD vectorizan los arrays y operan usando paralelización.

	Tiempo sin orden	fps_0	Tiempo con orden	fps_1	Aceleración al corregir índices (%)	Tiempo con compilación	fps_2	Aceleración al compilar (%)	Tiempo con paralelización	fps_3	Aceleración al paralelizar (%)
4k	34.226691	0.029217	34.622200	0.028883	-1.14	0.010019	99.809723	345463.22	0.002960	337.868858	238.51
8k	140.686359	0.007108	143.340304	0.006976	-1.85	0.040431	24.733191	354426.31	0.005996	166.771531	574.28
FHD	8.588898	0.116429	9.032742	0.110708	-4.91	0.004000	249.973419	225694.53	0.002030	492.693997	97.10
SD	3.728828	0.268181	3.969380	0.251929	-6.06	0.002000	500.036242	198383.38	0.000000	inf	inf
HD	0.938766	1.065228	0.980239	1.020159	-4.23	0.000000	inf	inf	0.000000	inf	NaN